MLOPS & CLOUD

AI · ML · DS / REVISION NOTES

COURSE NOTES

CI/CD Pipelines for Data & ML

Promoting Databricks code from dev to prod with Git-driven pipelines

2
SECTIONS

7
PAGES

Compiled October 2026
github.com/ankitgupta1729/AI-ML-DS

Contents

3 Merging/Branching Techniques 3

8 Creating the CI(Continuous Integration) Pipeline 4

1. Suppose, we have 2 environments: Dev and Prod (Instead of dev,uat,prod or dev,qa and prod). In Dev, we have databricks and different notebooks for data engineering task.

The main idea of CI/CD pipeline is to deploy code from one environment to another environment. In Prod, databricks is already installed but notebooks are not there.

To do CI/CD pipeline, we need a git repo. When we commit some changes in dev resource then it will be automatically deployed in prod.

2. What is CI/CD?

It is a set of practices used automate and streamline the process of building, testing and deploying code changes to different environments.

TOPIC 3

Merging/Branching Techniques

CI/CD pipeline will be triggered only when there is a change in the "main" branch.

So, if there are 2 data engineers then no data engineer should make changes in the main branch directly, instead they should create the copy of main branch say "feature1" and make changes in this branch and commit. Similarly, data engineer 2 can create the copy of main branch say "feature2" and make changes in this branch and commit.

Now, both data engineers should discuss and talk about priorities means whose change should be merged first in main branch.

Suppose, it is decided that data engineer 1 should merge first and then data engineer 2 should merge. So, first, data engineer 1 will create the merge(PR(pull request)) and once this PR will be approved then all the changes of data engineer 1 will be merged in main branch. After that, CI/CD pipeline will be triggered and it will deploy the changes of data engineer 1 in prod.

After that data engineer 2 will create the merge(PR(pull request)) and follow the same process.

4. First using your mail id, login to azure portal and create a resource group (`azure-data-engineering-project` for dev and `azure-data-engineering-project-prod` for prod) and create a resource as `azure-databricks` as azure databricks service in dev and launch a workspace and create a simple notebook name `test-hello-world.ipynb` in dev and create a resource as `azure-databricks-prod` for `azure-data-engineering-project-prod` .
   Now, using CI/CD pipeline deploy notebook in prod.

5. Now, we will create azure devops repo in `https://aex.dev.azure.com` . Go to `https://dev.azure.com/ankitgupta1729/` where `ankitgupta1729` represents the organization name.

Now, create a new project with project name as `Databricks CICD` . Now, in the left side, click on `Repo` and then in the upper side, create a new repository with name as `Databricks CICD Tutorial` with Git main branch.

Now, go to azure portal databricks where `test-hello-world.ipynb` file is there. Go to `Repos` folder from upper right side and click on `Create` option and then select `Repo` option.

Now, again go to azure devops site where readme file was there and clone the URL and put in as git repo url where we click on `Create` and then selected `Repo` .

Don't use PAT. Use MSFT ENTRA (https://claude.ai/share/4b7e9efc-1d33-4b22-bf60-17bbbef24257). Now, it will be reflected in `Repos` folder. Rename `Databricks%20CICD%20Tutorial` to `DatabricksCICDTutorial`.

6. Now, we have to protect the `main` branch. But before that move `test-hello-world.ipynb` notebook to `DatabricksCICDTutorial` folder.

Now, in the databricks workspace we have 2 files: `readme.md` and `test-hello-world.ipynb`. Now, click on `main` branch name and commit the changes by providing commit message and then click on `commit and push`.

Now, when your refresh `https://dev.azure.com/ankitgupta1729/Databricks%20CICD/_git/Databricks%20CICD%20Tutorial` then it will be reflected and new notebook file will be shown.

Now, to protect the `main` branch:

- Go to the azure devops portal (https://dev.azure.com/ankitgupta1729/Databricks%20CICD/_git/Databricks%20CICD%20Tutorial/branches) and select the `Branches` from left panel.
- The in front of main branch, click on three dots and select branch policies option.
- Enable `Require a minimum number of reviewers` and edit the `1` reviewers only in `Minimum number of reviewers` and enable `Allow requestors to approve their own changes` (not recommended but do it now so that you can approve merge requests).

Now, to test it, add a new notebook in `DatabricksCICDTutorial` folder with name as `new_notebook` and then again commit and push the changes in the same way by clicking on `main` branch name. Now, we will get error as `Error pushing changes Remote ref update was rejected. Make sure you have write access to this remote repository. Check the error details metadata for the more detailed Git error.` which shows that we have protected the `main` branch.

7. Now, create a new branch `feature-1` on the same page where we provide the commit message in the databricks workspace using `Create Branch` option. This new branch will be the copy of main branch and we can see the new notebook file in this branch again when we see the files in the `DatabricksCICDTutorial` folder as previous. Now, again commit and push the changes in the same way by clicking on `feature-1` branch name.

Now, we create a pull request to merge changes from `feature-1` branch to `main` branch.

- Go to Pull requests section in left side panel in azure devops portal as `https://dev.azure.com/ankitgupta1729/Databricks%20CICD/_git/Databricks%20CICD%20Tutorial/pullrequests?_a=mine` and click on `Create a pull request` option.
- Check the source and target branches and other tabs as `Files` which is important to identify new changes for merging.
- Select the approvers and approve and then complete it and see the changes in main branch.

TOPIC 8

Creating the CI(Continuous Integration) Pipeline

CI pipeline will copy all the files from `DatabricksCICDTutorial` folder to `live` folder which will be created automatically.

So, this `live` folder is the copy of main branch everytime and contains the latest files. This `live` folder will be used in adf(azure data factory) pipeline.

The CD pipeline will deploy `live` folder to databricks prod environment.

To organize all the files in `DatabricksCICDTutorial` folder we need to create a branch called `organize` from `main` branch. Here, organize means some notebook files are there and one readme file is there so need to organize the files in `DatabricksCICDTutorial` folder. So, click on `main` branch name and create branch `organize`.

Now, in `organize` branch with `DatabricksCICDTutorial` folder, create a new folder `Notebooks` and moved all notebook files in this folder.

Also create `extra` folder and moved `readme.md` file in this folder.

Now, we have to configure CI pipeline in such a way that only files in `Notebooks` folder will be deployed in the prod environment.

Now, commit and push the changes in the same way by clicking on `organize` branch name.

Now, refresh azure devops portal and create pull request to merge changes from `organize` branch to `main` branch. Now, approve and complete the pull request.

Now, we use the yaml file code to build CI/CD pipeline.

Now, to build logic in yaml file, first locally clone the azure devops repo main branch in vscode by given option. Now, in vscode, create a folder `CICD` and inside that put all files as shown.

Also, create a new branch in vscode by clicking on main in bottom left corner and create a new branch as `feature/ci-pipeline`.

9. Now, go to `cicd-pipelines.yml` file, it is the master file for CI/CD Pipeline. In this file,

- trigger is a property and main is its value. It means this yaml file will be triggered when there will be changes in main branch.
- Now, go to azure devops portal and from left side panel, click on `Piplines` and then click on `Library` and then select `Variable Group`. Now, put the `Variable Group Name` as `dbw-cicd-dev` from the `cicd-pipelines.yml`.

`pool` in the code `cicd-pipelines.yml` is simply the compute power. We can use microsoft hosted virtual machines.

`stages` is most important for deployment and it is for each environment.

Now, in azure devops portal, add variables as

dev-environment-name with value as `to be filled`

dev-resource-group-name with value as `rg-data-engineering-project`

dev-service-connection-name with value as `to be filled`

and save the variables group.

In azure devops, under pipelines, there is an option of `Environments`, click on it and create a new environment for dev as `dev-environment-databricks-cicd` with `None` as resource. And now update the dev-environment-name with value as `dev-environment-databricks-cicd` in Library tab.

Now, service connection is set so that CI/CD pipeline can use azure resources. For a particular resource group, there is access control(IAM) option is there, so click on it and you will see the information.

On azure devops portal, click on project settings (in bottom left corner), click on `service connections`. Now, select `Azure Resource Manager` and click on `Next` and then give the name of service connect as `dev-service-connection` and
save it. It will create a new service principal which will allow access for azure devops to azure resources.

Now, a new principal will be created and you can check in that dev resource group with Access Control(IAM) and role assignments.

Now, again update dev-service-connection-name with value as `dev-service-connection` in Library tab on azure devops portal.

Now, commit all the changes in `feature/ci-pipeline` branch in vscode with commit message as `added ci codes`.

Now, check the azure devops portal with the `feature/ci-pipeline` branch and see the changes.

10. Before creating the pipeline, we need to merge changes of `feature/ci-pipeline` branch to `main` branch because if we create the pipeline first and then merge the changes to main branch then pipeline would start immediately since there will be an update in the main branch. Now, merge changes of `feature/ci-pipeline` branch to `main` branch without deleting the `feature/ci-pipeline` branch.

Now, in azure devops portal, In Pipelines, select Pipelines option and click on `Create Pipeline`. Then click on `Azure Repos Git` and select the `DatabricksCICDTutorial` repo. Then in `Configure Your Pipeline`, select `Existing Azure Pipeline YAML file` and then select branch as `main` and Path as `CICD/cicd-pipelines.yml` file and click on `Continue`.

Now, first `Save` the pipeline and don't run it. Now, we have to give certain permissions to this pipeline. Go to `Environments` tab from left side panel, and then select `dev-environment-databricks-cicd` and then from upper right corner, click on three dots and select `Security` and then in `Pipeline permissions`, click on "+" symbol and select `DatabricksCICDTutorial`.

Now, we give permission for pipeline so that it can read variable groups, for that go to `Library` and then select variable group and then from upper side, select `Pipeline permissions` and then in `Pipeline permissions`, click on "+" symbol and select `DatabricksCICDTutorial`.

Also, we need to pipeline permissions to service connection. So, go to `Project Settings` and then click on `Service Connections` and then select `dev-service-connection` and then from three dots, select `Security` and then in `Pipeline permissions`, click on "+" symbol and select `DatabricksCICDTutorial`.

11. Now, to test it, go to databricks workspace and then open `Notebooks` folder and then from main branch, create a new branch `testing-ci-pipeline` and Now, add a new notebook as `testing-ci-notebook` and then Commit and push changes and then create a new pull request from `testing-ci-pipeline` branch to `main` branch in azure devops portal.

Now, go to Azure Devops portal and select `Pipelines` in `Pipelines` tab and then check the status of running pipeline.

[For dev testing, don't use anything related to prod in yaml file]

[Make sure Add that SP to the workspace as an admin.
In your Databricks workspace (adb-7405605083397751.11.azuredatabricks.net):

Top-right (your avatar) → Settings → Identity and access.

Click Add service principal again.
On the first screen of the dialog, do NOT pick "Add new." Look for the option to add an existing service principal (a search box for existing account principals). If you're already on the "Management / Microsoft Entra ID

managed" form, click Back to get to that first screen.
Search by the Application ID 4c02c10c-9988-4595-918c-dd8d9c60f0e1 (or by name). Select the existing principal that appears.
Turn Admin access → On, then Add. This assigns the already-existing account SP to the workspace as an admin (no new account SP is created, so no collision).
]

12. Now, we focus on CD(Continuous Deployment/Delivery) part. We deploy the changes from main branch of dev environment to live folder of prod environment.

First, in vscode, open the cicd-pipelines.yml file with main branch and create a new branch called `feature/cd-pipeline` from main branch.

Here, we have to add prod related group and stages in cicd-pipelines.yml file.

Now, we have to add variable group,environment and service connection for prod in azure devops.

Add a new variable group as `dbw-cicd-prod` and add variables as

- `prod-environment-name` with value as `prod-environment-databricks-cicd`
- `prod-resource-group-name` with value as `azure-data-engineering-project-prod`
- `prod-service-connection-name` with value as `prod-service-connection`

Now, create the new environment with name as `prod-environment-databricks-cicd` in pipelines tab on azure devops for prod.

Now, create a new service connection with name as `prod-service-connection` in pipelines tab on azure devops for prod.

AAt the end give pipeline permissions for resource group,environments and service connection as we did for dev.

Now, commit all the changes in `feature/cd-pipeline` branch in vscode with commit message as `added cd pipeline` .

Now, create a new pull request from `feature/cd-pipeline` branch to `main` branch in azure devops portal.

After that pipeline will automatically triggered and we can see in the pipelines tab and build will be having 2 stages.

13. On Azure devops portal, there is an option of `Approval and checks` and it is used between dev and prod pipeline. When dev got succeeded and then we need some high level approval like tech lead and then we need to deploy the changes to prod environment.

For this, on azure devops portal, go to prod environment in pipelines tab and then go to `Approval and checks` and select `Approval` from + symbol. Then add the mail id of a person from which you need approval and then a mail will be triggered to that person and in this way prod environment will be protected.

Now, for full cicd testing,

go to dev workspace and add a new branch called `end-to-end-cicd-testing` from main branch. Now, create a new notebook `final-cicd-test-notebook` and commit and push the changes.

Now, create a new pull request from `end-to-end-cicd-testing` branch to `main` branch in azure devops portal.

Now, go to azure devops portal and check the status of running pipeline.